

Experiment Name:

Unit Testing with Assertion based testing

Objectives:

- To Implement Functional Unit Testing
- To Demonstrate Assertion-Based Verification
- Validate Error Handling and Edge Cases
- To Practice Modular Software Architecture
- To Automate Software Reliability Checks

Theory:

Unit testing is a fundamental practice in software engineering where individual components of a software application are tested in isolation to verify their functional correctness. In object-oriented programming, a "unit" typically refers to the smallest testable part of the system, such as a specific method, function, or class. By focusing on these granular elements, developers can validate the internal logic of an application—such as ensuring an addition function correctly calculates the sum of two integers—without the interference of external dependencies like databases or user interfaces. This isolation is often achieved through techniques like mocking, which simulates external systems to ensure that any failure is strictly due to a flaw in the unit's logic.

Required Equipment / Software:

- A C++ compiler and development environment (IDE).
- `<cassert>` header file

Procedure followed:

1. Environment Setup:

- Launched VS Code as the IDE
- Verified that the MinGW GCC compiler was correctly installed and configured in the system path to enable C++ compilation.

2. Implementation of the Source Code (Unit Under Test):

- Created a C++ source file named `Calculator.cpp`.
- Defined a class named `Calculator` containing public member functions for arithmetic operations: `add ()`, `sub ()`, `mult ()`, `div ()` and `modulus ()`.
- Implemented errors handling logic within the `div ()` function to explicitly check if the divisor is zero.
- Configured the function to throw a string exception if an invalid division was attempted, preventing a runtime crash.

3. Implementation of the Test Driver:

- Created a separate test file named `test.cpp`.

- Included the necessary standard libraries and the source file to link the testing script with the calculator logic.
- Wrote a main () function to serve as the entry point for testing.

4. Development of Test Cases:

- Positive Testing: Wrote assert () statements to verify correct outputs for valid inputs.
- Edge Case Testing: Implemented a try-catch block specifically to test the division-by-zero scenario.
- Verification: Used a Boolean flag to assert that the custom exception was successfully thrown and caught when dividing by zero.

5. Compilation and Execution:

- Opened the integrated terminal in VS Code.
- Executed the compilation command for test.cpp file.
- Run the resulting executable.
- Verified the output in the terminal to ensure all assertions passed and the "SUCCESS" message was displayed.

Source Code:

File name→ “Calculator.cpp”

```
#include<bits/stdc++.h>
using namespace std;

class Calculator
{
public:
float add(float a, float b)
{return a+b;}

float sub(float a, float b)
{return a-b;}

float mult(float a, float b)
{return a*b;}

float div(float a, float b)
{
if(b==0)
{
throw string("Zero division not possible");
}
return a/b;
}
```

```
int modulus(int a,int b)
{return a%b;}
};
```

Testing Code:

File name→ “test.cpp”

```
#include <iostream>
#include <cassert>
#include "Calculator.cpp"
using namespace std;

int main() {
    Calculator calc;

    cout<<"Starting Unit Tests..."<<endl;

    // --- Test 1: Addition ---
    cout<<"Testing Addition... ";
    assert(calc.add(10.5,20)==30.5);
    assert(calc.add(-5,5)==0);
    cout<<"OK!"<<endl;

    // --- Test 2: Subtraction ---
    cout << "Testing Subtraction... ";
    assert(calc.sub(10,5)==5);
    assert(calc.sub(10,20)==-10);
    cout<<"OK!"<<endl;

    // --- Test 3: Multiplication ---
    cout<<"Testing Multiplication... ";
    assert(calc.mult(5,4)==20);
    assert(calc.mult(10,2)==20);
    cout<<"OK!"<<endl;

    // --- Test 4: Division ---
    cout<<"Testing Division... ";
    assert(calc.div(10,2)==5);
    assert(calc.div(9,3)==3);
    cout<<"OK!"<<endl;

    cout<<"Testing Division by Zero... ";
    bool exceptionCaught=false;
```

```

try
{
    calc.div(10,0);
}
catch (string e)
{
    exceptionCaught=true;
}
assert(exceptionCaught==true);
cout<<"OK!"<<endl;

// --- Test 5: Modulus ---
cout<<"Testing Modulus... ";
assert(calc.modulus(10,2)==0);
assert(calc.modulus(9,7)==2);
cout<<"OK!"<<endl;

cout<<"\n-----"<<endl;
cout<<"SUCCESS: All tests passed!" <<endl;
cout<< "-----"<<endl;

return 0;
}

```

Output:

```

Starting Unit Tests...
Testing Addition... OK!
Testing Subtraction... OK!
Testing Multiplication... OK!
Testing Division... OK!
Testing Division by Zero... OK!
Testing Modulus... OK!

```

```

-----
SUCCESS: All tests passed!
-----

```

```

Press any key to continue . . . |

```

Fig- For all the correct Test

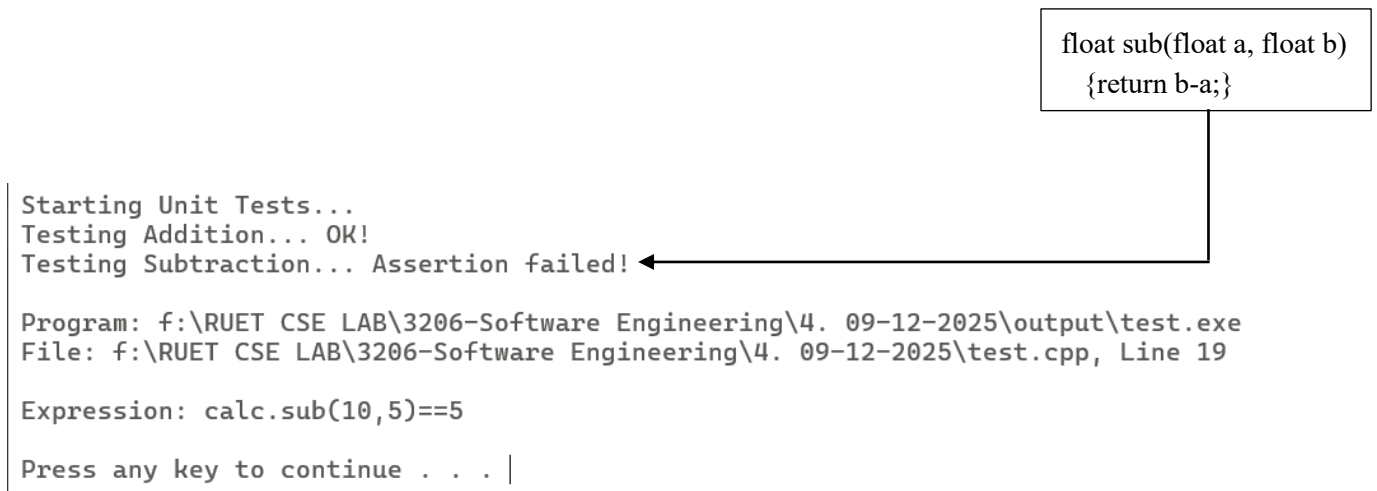


Fig- For any Incorrect Test

Conclusion:

In this experiment, we successfully implemented and verified the functionality of a modular Calculator class using Unit Testing principles. By employing an Assertion-Based Testing strategy (using the C++ <cassert> library), we created an automated test harness that validated the correctness of arithmetic operations including addition, subtraction, multiplication, division and modulus.